

SOLID Principles for OOP and LLD

SOLID is a set of five object-oriented design principles that help you build code that is easy to understand, change, test, and extend. These principles are especially important in Low Level Design interviews because interviewers are usually not only checking whether your classes work, but also whether your design can survive changing requirements.

SOLID stands for:

1. Single Responsibility Principle
2. Open/Closed Principle
3. Liskov Substitution Principle
4. Interface Segregation Principle
5. Dependency Inversion Principle

The goal of SOLID is not to create more classes for the sake of it. The goal is to put responsibilities in the right place so that changes stay localized.

Why SOLID Matters in Placements

In OOP and LLD interviews, you are often asked to design systems like:

- Parking lot
- Elevator system
- Library management system
- Splitwise
- BookMyShow
- Chess
- Snake and Ladder
- ATM
- Vending machine
- Food delivery system
- Ride sharing system
- Logging framework
- Notification system

For these questions, a working design is not enough. A good design should answer:

- What happens when requirements change?
- Can we add new features without breaking existing code?
- Are classes doing too much?
- Can the code be tested easily?
- Are abstractions meaningful?
- Are subclasses truly replaceable?
- Are high-level modules independent of low-level implementation details?

SOLID helps you reason about these questions.

1. Single Responsibility Principle

Definition

A class should have only one reason to change.

In simpler words, a class should do one main job. It should not mix unrelated responsibilities.

Important Clarification

Single Responsibility does not mean a class should have only one method. It means all methods in the class should support one clear responsibility.

For example, a `User` class can have methods like:

- `getName()`
- `getEmail()`
- `changePassword()`

These may still belong together if the responsibility is managing user state.

But if `User` also has:

- `saveToDatabase()`
- `sendWelcomeEmail()`
- `generateInvoice()`
- `renderUserProfileHtml()`

then it is doing too much.

Bad Example

```
class Invoice {
    private double amount;

    public double calculateTotal() {
        return amount + amount * 0.18;
    }

    public void saveToDatabase() {
        // database logic
    }

    public void printInvoice() {
        // printing logic
    }

    public void sendEmail() {
        // email logic
    }
}
```

```
}  
}
```

This class has many reasons to change:

- Tax calculation changes
- Database changes
- Print format changes
- Email service changes

That violates SRP.

Better Design

```
class Invoice {  
    private double amount;  
  
    public double calculateTotal() {  
        return amount + amount * 0.18;  
    }  
}  
  
class InvoiceRepository {  
    public void save(Invoice invoice) {  
        // database logic  
    }  
}  
  
class InvoicePrinter {  
    public void print(Invoice invoice) {  
        // printing logic  
    }  
}  
  
class InvoiceEmailService {  
    public void send(Invoice invoice) {  
        // email logic  
    }  
}
```

Now each class has one main reason to change.

How to Identify SRP Violations

Look for class names that contain words like:

- Manager
- Processor
- Handler

- Utility
- Helper
- Service

These names are not always wrong, but they are often signs that a class may be doing too much.

Also look for methods that belong to different domains:

```
class OrderManager {  
    public void calculatePrice() {}  
    public void saveOrder() {}  
    public void sendSms() {}  
    public void updateInventory() {}  
    public void generatePdf() {}  
}
```

This class mixes pricing, persistence, notification, inventory, and document generation.

SRP in LLD Examples

Parking Lot

Bad:

```
class ParkingLot {  
    public void parkVehicle() {}  
    public void calculateFee() {}  
    public void printTicket() {}  
    public void processPayment() {}  
}
```

Better:

- **ParkingLot** manages floors and spots
- **ParkingSpot** represents a spot
- **Ticket** stores parking session data
- **FeeCalculator** calculates fee
- **PaymentService** handles payment
- **TicketPrinter** prints ticket

BookMyShow

Bad:

```
class BookingService {  
    public void searchMovies() {}  
    public void reserveSeats() {}  
}
```

```
public void processPayment() {}  
public void sendNotification() {}  
public void generateInvoice() {}  
}
```

Better:

- `MovieCatalogService`
- `SeatLockService`
- `BookingService`
- `PaymentService`
- `NotificationService`
- `InvoiceService`

Interview Explanation

You can say:

"I am applying Single Responsibility Principle by separating business logic, persistence, payment, and notification responsibilities into different classes. This keeps each class focused and makes future changes less risky."

Benefits

- Easier to understand classes
- Easier to test
- Less merge conflict in teams
- Changes are localized
- Fewer side effects
- Better reusability

Common Mistake

Do not create too many tiny classes without purpose. SRP is about reasons to change, not blindly splitting every method into a new class.

2. Open/Closed Principle

Definition

Software entities should be open for extension but closed for modification.

This means you should be able to add new behavior without changing existing, tested code.

Simple Meaning

When a new requirement comes, you should prefer adding a new class or implementation instead of modifying a big chain of `if-else` or `switch` statements.

Bad Example

```
class DiscountCalculator {  
    public double calculate(String customerType, double amount) {  
        if (customerType.equals("REGULAR")) {  
            return amount * 0.05;  
        } else if (customerType.equals("PREMIUM")) {  
            return amount * 0.10;  
        } else if (customerType.equals("VIP")) {  
            return amount * 0.20;  
        }  
        return 0;  
    }  
}
```

Problem:

Every time a new customer type is added, this class must be modified.

That violates OCP.

Better Design

```
interface DiscountStrategy {  
    double calculate(double amount);  
}  
  
class RegularDiscount implements DiscountStrategy {  
    public double calculate(double amount) {  
        return amount * 0.05;  
    }  
}  
  
class PremiumDiscount implements DiscountStrategy {  
    public double calculate(double amount) {  
        return amount * 0.10;  
    }  
}  
  
class VipDiscount implements DiscountStrategy {  
    public double calculate(double amount) {  
        return amount * 0.20;  
    }  
}  
  
class DiscountCalculator {  
    public double calculate(DiscountStrategy strategy, double amount) {  
        return strategy.calculate(amount);  
    }  
}
```

```
}  
}
```

Now a new discount can be added by creating a new class:

```
class FestivalDiscount implements DiscountStrategy {  
    public double calculate(double amount) {  
        return amount * 0.30;  
    }  
}
```

Existing code does not need to change.

OCP and Polymorphism

OCP is usually achieved using:

- Interfaces
- Abstract classes
- Strategy pattern
- Factory pattern
- Template method pattern
- Observer pattern
- Decorator pattern

OCP in LLD Examples

Notification System

Bad:

```
class NotificationService {  
    public void send(String type, String message) {  
        if (type.equals("EMAIL")) {  
            // send email  
        } else if (type.equals("SMS")) {  
            // send sms  
        } else if (type.equals("PUSH")) {  
            // send push notification  
        }  
    }  
}
```

Better:

```
interface NotificationSender {
    void send(String message);
}

class EmailSender implements NotificationSender {
    public void send(String message) {
        // send email
    }
}

class SmsSender implements NotificationSender {
    public void send(String message) {
        // send sms
    }
}

class PushNotificationSender implements NotificationSender {
    public void send(String message) {
        // send push notification
    }
}
```

Now WhatsApp notification can be added without modifying existing sender classes.

Parking Fee Calculation

Different vehicles can have different pricing:

- Bike fee
- Car fee
- Truck fee
- Electric vehicle fee
- Weekend fee
- Mall parking fee
- Airport parking fee

Instead of writing a huge `if-else`, use:

```
interface FeeStrategy {
    double calculateFee(Ticket ticket);
}
```

Then create:

- HourlyFeeStrategy
- FlatRateFeeStrategy
- VehicleBasedFeeStrategy
- WeekendFeeStrategy

Interview Explanation

You can say:

"I am keeping the design open for extension by depending on abstractions. For example, if a new payment method or notification channel is added, we can create a new implementation without changing the existing service logic."

Benefits

- Easier feature addition
- Less risk of breaking tested code
- Avoids large conditional blocks
- Supports plugin-like design
- Makes use of polymorphism

Common Mistake

Do not apply OCP everywhere from the beginning. If there is only one behavior and no realistic variation, a simple class is enough. OCP is most useful where change is likely.

3. Liskov Substitution Principle

Definition

Objects of a superclass should be replaceable with objects of a subclass without breaking the correctness of the program.

In simple words:

If class **B** extends class **A**, then we should be able to use **B** wherever **A** is expected, without unexpected behavior.

Simple Example

If a method expects a **Vehicle**, then passing a **Car**, **Bike**, or **Truck** should work correctly if they are valid subtypes of **Vehicle**.

```
class Vehicle {  
    public void startEngine() {}  
}  
  
class Car extends Vehicle {  
    public void startEngine() {  
        // start car engine  
    }  
}
```

This looks fine.

But what about:

```
class Bicycle extends Vehicle {
    public void startEngine() {
        throw new UnsupportedOperationException("Bicycle has no engine");
    }
}
```

This violates LSP because `Bicycle` cannot truly behave like a `Vehicle` if `Vehicle` promises engine behavior.

Better Design

```
interface Vehicle {
    void move();
}

interface EngineVehicle extends Vehicle {
    void startEngine();
}

class Car implements EngineVehicle {
    public void move() {}
    public void startEngine() {}
}

class Bicycle implements Vehicle {
    public void move() {}
}
```

Now the abstraction is correct.

Classic Rectangle-Square Problem

Mathematically, a square is a rectangle. But in OOP design, inheritance depends on behavior, not just real-world classification.

Bad design:

```
class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(int width) {
        this.width = width;
    }
}
```

```
    public void setHeight(int height) {
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}

class Square extends Rectangle {
    public void setWidth(int width) {
        this.width = width;
        this.height = width;
    }

    public void setHeight(int height) {
        this.width = height;
        this.height = height;
    }
}
```

Problem:

Code expecting a rectangle may do this:

```
void test(Rectangle rectangle) {
    rectangle.setWidth(5);
    rectangle.setHeight(10);
    assert rectangle.getArea() == 50;
}
```

This works for `Rectangle`, but fails for `Square`.

So `Square` is not safely substitutable for `Rectangle`.

Correct Thinking

Inheritance should be based on behavior, not only on real-world "is-a" relationships.

Ask:

- Can the subclass honor all promises of the parent?
- Does the subclass throw exceptions for parent methods?
- Does the subclass weaken expected behavior?
- Does the subclass surprise callers?

If yes, inheritance may be wrong.

Signs of LSP Violation

- Subclass overrides a method and throws `UnsupportedOperationException`
- Subclass leaves inherited method empty
- Subclass changes method behavior in a surprising way
- Code checks object type using `instanceof`
- Parent class has methods that do not apply to all children
- Subclass requires stronger preconditions than parent
- Subclass provides weaker postconditions than parent

LSP in LLD Examples

Payment System

Bad:

```
abstract class Payment {
    abstract void pay(double amount);
    abstract void refund(double amount);
}

class CashPayment extends Payment {
    void pay(double amount) {}

    void refund(double amount) {
        throw new UnsupportedOperationException();
    }
}
```

If cash payments cannot support the same refund behavior, the abstraction is wrong.

Better:

```
interface Payable {
    void pay(double amount);
}

interface Refundable {
    void refund(double amount);
}

class CardPayment implements Payable, Refundable {
    public void pay(double amount) {}
    public void refund(double amount) {}
}

class CashPayment implements Payable {
    public void pay(double amount) {}
}
```

Bird Example

Bad:

```
class Bird {
    void fly() {}
}

class Penguin extends Bird {
    void fly() {
        throw new UnsupportedOperationException();
    }
}
```

Better:

```
interface Bird {
    void eat();
}

interface FlyingBird extends Bird {
    void fly();
}

class Sparrow implements FlyingBird {
    public void eat() {}
    public void fly() {}
}

class Penguin implements Bird {
    public void eat() {}
}
```

Interview Explanation

You can say:

"I am avoiding inheritance where the child cannot fully honor the parent contract. Instead, I split the abstraction so each implementation supports only the behavior it can correctly provide."

Benefits

- Prevents surprising runtime errors
- Makes polymorphism reliable
- Improves correctness of inheritance
- Reduces `instanceof` checks
- Encourages better abstraction design

Common Mistake

Do not assume real-world hierarchy automatically becomes code hierarchy. In OOP, behavior matters more than taxonomy.

4. Interface Segregation Principle

Definition

Clients should not be forced to depend on interfaces they do not use.

In simple words:

Do not create large, fat interfaces. Prefer smaller, role-specific interfaces.

Bad Example

```
interface Machine {
    void print();
    void scan();
    void fax();
}

class OldPrinter implements Machine {
    public void print() {
        // print
    }

    public void scan() {
        throw new UnsupportedOperationException();
    }

    public void fax() {
        throw new UnsupportedOperationException();
    }
}
```

`OldPrinter` is forced to implement methods it does not support.

This violates ISP.

Better Design

```
interface Printer {
    void print();
}

interface Scanner {
```

```
        void scan();
    }

    interface FaxMachine {
        void fax();
    }

    class OldPrinter implements Printer {
        public void print() {}
    }

    class MultiFunctionPrinter implements Printer, Scanner, FaxMachine {
        public void print() {}
        public void scan() {}
        public void fax() {}
    }
```

Now each class implements only what it needs.

ISP vs SRP

SRP is about classes having one reason to change.

ISP is about interfaces being small and client-specific.

They are related, but not the same.

Fat Interface Example in LLD

Bad:

```
interface UserActions {
    void createPost();
    void deletePost();
    void banUser();
    void makePayment();
    void refundPayment();
    void viewAnalytics();
}
```

This interface mixes actions for:

- Normal users
- Admins
- Payment users
- Analysts

Better:

```
interface ContentCreator {  
    void createPost();  
    void deleteOwnPost();  
}  
  
interface Admin {  
    void banUser();  
    void deleteAnyPost();  
}  
  
interface PayingCustomer {  
    void makePayment();  
}  
  
interface Analyst {  
    void viewAnalytics();  
}
```

ISP in LLD Examples

Food Delivery System

Bad:

```
interface DeliveryPartner {  
    void acceptOrder();  
    void pickUpFood();  
    void deliverFood();  
    void cookFood();  
    void processPayment();  
}
```

This interface mixes delivery, cooking, and payment.

Better:

- OrderAcceptor
- FoodPicker
- FoodDeliverer
- RestaurantPartner
- PaymentProcessor

Parking Lot

Bad:

```
interface ParkingOperations {  
    void parkVehicle();  
    void unparkVehicle();  
    void calculateFee();  
    void processPayment();  
    void generateReport();  
    void addParkingFloor();  
}
```

Better:

- ParkingService
- FeeCalculator
- PaymentProcessor
- ParkingAdminService
- ReportGenerator

Interview Explanation

You can say:

"I am applying Interface Segregation by creating smaller interfaces for specific roles. This prevents classes from implementing methods they do not need and keeps the design flexible."

Benefits

- Avoids unnecessary dependencies
- Reduces dummy method implementations
- Improves testability
- Makes APIs easier to understand
- Supports role-based design
- Helps avoid LSP violations

Common Mistake

Do not create one-method interfaces everywhere without reason. ISP is useful when clients need different subsets of behavior.

5. Dependency Inversion Principle

Definition

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Also:

Abstractions should not depend on details. Details should depend on abstractions.

Simple Meaning

Business logic should not directly depend on concrete classes like:

- `MySQLDatabase`
- `GmailEmailSender`
- `RazorpayPaymentGateway`
- `FileLogger`

Instead, business logic should depend on interfaces like:

- `UserRepository`
- `EmailSender`
- `PaymentGateway`
- `Logger`

Concrete classes implement those interfaces.

Bad Example

```
class OrderService {  
    private MySQLOrderRepository repository = new MySQLOrderRepository();  
    private EmailService emailService = new EmailService();  
  
    public void placeOrder(Order order) {  
        repository.save(order);  
        emailService.sendEmail("Order placed");  
    }  
}
```

Problem:

`OrderService` is tightly coupled to `MySQLOrderRepository` and `EmailService`.

If we switch to PostgreSQL, MongoDB, or a mock repository for tests, we must change `OrderService`.

Better Design

```
interface OrderRepository {  
    void save(Order order);  
}  
  
interface NotificationService {  
    void send(String message);  
}  
  
class MySQLOrderRepository implements OrderRepository {  
    public void save(Order order) {  
        // save to MySQL  
    }  
}
```

```
}  
}  
  
class EmailNotificationService implements NotificationService {  
    public void send(String message) {  
        // send email  
    }  
}  
  
class OrderService {  
    private final OrderRepository repository;  
    private final NotificationService notificationService;  
  
    public OrderService(OrderRepository repository, NotificationService  
notificationService) {  
        this.repository = repository;  
        this.notificationService = notificationService;  
    }  
  
    public void placeOrder(Order order) {  
        repository.save(order);  
        notificationService.send("Order placed");  
    }  
}
```

Now `OrderService` depends on abstractions, not concrete details.

Dependency Injection

Dependency Inversion is the principle.

Dependency Injection is one way to implement it.

Example:

```
OrderRepository repository = new MySqlOrderRepository();  
NotificationService notificationService = new EmailNotificationService();  
  
OrderService service = new OrderService(repository, notificationService);
```

Here, dependencies are provided from outside the class.

Types of Dependency Injection

Constructor Injection

```
class OrderService {  
    private final OrderRepository repository;
```

```
public OrderService(OrderRepository repository) {  
    this.repository = repository;  
}  
}
```

Best for required dependencies.

Setter Injection

```
class OrderService {  
    private OrderRepository repository;  
  
    public void setRepository(OrderRepository repository) {  
        this.repository = repository;  
    }  
}
```

Useful when dependency is optional or can change.

Method Injection

```
class OrderService {  
    public void placeOrder(Order order, PaymentGateway paymentGateway) {  
        paymentGateway.pay(order.getAmount());  
    }  
}
```

Useful when dependency is needed only for one operation.

DIP in LLD Examples

Payment System

Bad:

```
class CheckoutService {  
    private RazorpayGateway gateway = new RazorpayGateway();  
  
    public void checkout(double amount) {  
        gateway.pay(amount);  
    }  
}
```

Better:

```
interface PaymentGateway {
    void pay(double amount);
}

class RazorpayGateway implements PaymentGateway {
    public void pay(double amount) {}
}

class StripeGateway implements PaymentGateway {
    public void pay(double amount) {}
}

class CheckoutService {
    private final PaymentGateway paymentGateway;

    public CheckoutService(PaymentGateway paymentGateway) {
        this.paymentGateway = paymentGateway;
    }

    public void checkout(double amount) {
        paymentGateway.pay(amount);
    }
}
```

Now we can use Razorpay, Stripe, PayPal, UPI, or a fake gateway for testing.

Logger

Bad:

```
class UserService {
    private FileLogger logger = new FileLogger();
}
```

Better:

```
interface Logger {
    void log(String message);
}

class FileLogger implements Logger {
    public void log(String message) {}
}

class ConsoleLogger implements Logger {
    public void log(String message) {}
}
```

```
class UserService {  
    private final Logger logger;  
  
    public UserService(Logger logger) {  
        this.logger = logger;  
    }  
}
```

Interview Explanation

You can say:

"I am making high-level business services depend on interfaces instead of concrete implementations. This allows us to replace databases, payment gateways, notification providers, and loggers without changing business logic."

Benefits

- Loose coupling
- Easier unit testing
- Easier replacement of implementations
- Supports mocking
- Improves maintainability
- Keeps business logic independent of infrastructure

Common Mistake

Do not confuse Dependency Inversion with simply using an interface everywhere. DIP is useful when a high-level policy should not be controlled by low-level details.

SOLID Together: Example Design

Let us design a simple notification system.

Requirements

- Send notifications to users
- Support email and SMS
- Add push notification later
- Store notification history
- Retry failed notifications

Bad Design

```
class NotificationManager {  
    public void send(String type, String message) {
```

```
        if (type.equals("EMAIL")) {
            // send email
        } else if (type.equals("SMS")) {
            // send sms
        }

        // save to database
        // retry logic
        // logging
    }
}
```

Problems:

- Violates SRP: sending, persistence, retry, and logging are mixed
- Violates OCP: new notification type requires modifying existing code
- May violate DIP: depends directly on concrete email/SMS logic
- Can lead to ISP issues if one large interface is created

Better Design

```
interface NotificationChannel {
    void send(Notification notification);
}

class EmailChannel implements NotificationChannel {
    public void send(Notification notification) {}
}

class SmsChannel implements NotificationChannel {
    public void send(Notification notification) {}
}

interface NotificationRepository {
    void save(Notification notification);
}

class NotificationService {
    private final NotificationChannel channel;
    private final NotificationRepository repository;

    public NotificationService(NotificationChannel channel, NotificationRepository repository) {
        this.channel = channel;
        this.repository = repository;
    }

    public void notify(Notification notification) {
        channel.send(notification);
        repository.save(notification);
    }
}
```

```
}  
}
```

SOLID usage:

- SRP: separate channel, repository, service
- OCP: add `PushChannel` without modifying `NotificationService`
- LSP: every `NotificationChannel` must be substitutable
- ISP: channel interface has only required behavior
- DIP: service depends on abstractions

SOLID and Design Patterns

SOLID principles are not design patterns, but design patterns often help apply SOLID.

Strategy Pattern

Useful for OCP.

Examples:

- Payment strategy
- Discount strategy
- Fee calculation strategy
- Route calculation strategy
- Sorting strategy

Factory Pattern

Useful when object creation logic becomes complex.

Examples:

- Create payment gateway based on payment type
- Create vehicle object based on vehicle type
- Create notification sender based on channel

Observer Pattern

Useful for decoupling event producers and listeners.

Examples:

- Order placed event triggers email, inventory update, invoice generation
- Stock price update notifies subscribers
- Chat message notifies connected users

Decorator Pattern

Useful for extending behavior without modifying existing classes.

Examples:

- Add extra toppings to pizza
- Add insurance or gift wrap to order
- Add compression/encryption to data stream

Adapter Pattern

Useful for DIP when integrating third-party APIs.

Example:

```
interface PaymentGateway {  
    void pay(double amount);  
}  
  
class RazorpayAdapter implements PaymentGateway {  
    private RazorpayClient client;  
  
    public void pay(double amount) {  
        client.makePayment(amount);  
    }  
}
```

Your business logic depends on `PaymentGateway`, not directly on `RazorpayClient`.

Common Interview Questions

What is SOLID?

SOLID is a set of five object-oriented design principles that improve maintainability, extensibility, and testability. The five principles are Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.

Is SOLID only for Java?

No. SOLID is language-independent, but it is most commonly discussed in object-oriented languages like Java, C++, C#, Python, and TypeScript.

Difference between OCP and DIP?

OCP says code should be open for extension and closed for modification.

DIP says high-level modules should depend on abstractions, not concrete low-level modules.

DIP often helps achieve OCP.

Difference between SRP and ISP?

SRP is about class responsibilities.

ISP is about interface responsibilities.

SRP asks: "Does this class have more than one reason to change?"

ISP asks: "Is this interface forcing clients to depend on methods they do not use?"

Difference between LSP and polymorphism?

Polymorphism allows different objects to be used through a common interface.

LSP ensures that this substitution is behaviorally correct.

Without LSP, polymorphism can become unsafe.

Is inheritance bad?

No. Inheritance is useful when there is a true behavioral relationship. But composition is often safer when behavior varies independently.

Prefer inheritance when:

- The child truly is a behavioral subtype
- The parent contract applies completely to the child
- You want polymorphic behavior

Prefer composition when:

- You only want to reuse code
- Behavior needs to be combined flexibly
- Subclasses would override too much

How does SOLID help testing?

SOLID makes testing easier because:

- Small classes are easier to test
- Dependencies can be mocked
- Business logic is separated from infrastructure
- Interfaces allow fake implementations
- Fewer unrelated side effects happen during tests

How to Explain SOLID in an LLD Interview

A strong answer should not just list definitions. Show how you apply the principles.

Example:

"For the parking lot design, I will keep fee calculation separate from parking spot allocation. That follows SRP. I will define a `FeeStrategy` interface so we can add hourly, flat-rate, or vehicle-based pricing without changing the parking service, which follows OCP. I will avoid forcing all vehicle types to implement behavior they do not support, which protects LSP and ISP. Finally, the main services will depend on abstractions like `PaymentProcessor` and `TicketRepository`, not concrete implementations, which follows DIP."

This kind of answer sounds much stronger than simply saying:

"I used SOLID principles."

Quick Revision Table

Principle	Main Question	Bad Sign	Common Fix
SRP	Does this class have one reason to change?	One class handles business logic, DB, email, logging	Split responsibilities
OCP	Can I add behavior without modifying old code?	Large <code>if-else</code> or <code>switch</code> on type	Use polymorphism/strategy
LSP	Can child replace parent safely?	Child throws unsupported exception	Fix hierarchy or split interface
ISP	Are clients forced to implement unused methods?	Fat interface with dummy methods	Smaller role-based interfaces
DIP	Does high-level logic depend on concrete details?	<code>new MySqlRepo()</code> inside service	Depend on interfaces and inject dependencies

Practical Checklist for LLD

Before finalizing any design, ask:

- Does each class have a clear responsibility?
 - Are there any classes doing persistence, business logic, and communication together?
 - Are there large conditional blocks based on type?
 - Can new payment methods, notification channels, pricing rules, or vehicle types be added easily?
 - Are subclasses truly substitutable for parents?
 - Are any methods throwing `UnsupportedOperationException`?
 - Are interfaces too large?
 - Are services directly creating concrete dependencies?
 - Can I unit test the main business logic without real database, network, or payment gateway?
 - Are abstractions based on actual variation, not imaginary future requirements?
-

High-Value Placement Examples

Parking Lot

Likely SOLID abstractions:

- `Vehicle`
- `ParkingSpot`
- `ParkingFloor`
- `Ticket`
- `SpotAllocationStrategy`
- `FeeCalculationStrategy`
- `PaymentProcessor`
- `TicketRepository`

SOLID explanation:

- SRP: allocation, fee calculation, and payment are separate
- OCP: new fee strategies can be added
- LSP: different vehicles must behave consistently as vehicles
- ISP: payment, allocation, and reporting interfaces stay separate
- DIP: services depend on strategy interfaces

Elevator System

Likely SOLID abstractions:

- `Elevator`
- `ElevatorController`
- `Request`
- `SchedulingStrategy`
- `Door`
- `Display`

SOLID explanation:

- SRP: scheduling is separate from elevator movement
- OCP: add new scheduling algorithms like nearest elevator or load-based scheduling
- DIP: controller depends on `SchedulingStrategy`, not a concrete scheduler

Vending Machine

Likely SOLID abstractions:

- `VendingMachine`
- `State`
- `Product`
- `Inventory`
- `PaymentProcessor`

- `DispenseService`

SOLID explanation:

- SRP: inventory, payment, and dispensing are separate
- OCP: new payment methods can be added
- State pattern helps keep behavior clean
- DIP: vending machine depends on payment abstraction

Splitwise

Likely SOLID abstractions:

- `Expense`
- `Split`
- `User`
- `Group`
- `SplitStrategy`
- `BalanceSheet`
- `SettlementStrategy`

SOLID explanation:

- SRP: expense creation, split calculation, and settlement are separate
- OCP: equal, exact, and percentage split strategies can be added
- DIP: expense service depends on split strategy abstraction

Chess

Likely SOLID abstractions:

- `Board`
- `Cell`
- `Piece`
- `Move`
- `MoveValidator`
- `Player`
- `Game`

SOLID explanation:

- SRP: board state, move validation, and game flow are separate
- OCP: each piece can define its own movement logic
- LSP: every chess piece should fit the `Piece` contract

Common Anti-Patterns Against SOLID

God Class

One class does everything.

Example:

```
class AppManager {  
    // login, payment, database, notification, logging, reporting  
}
```

Usually violates SRP.

Type Checking Everywhere

```
if (vehicle.type == CAR) {}  
else if (vehicle.type == BIKE) {}  
else if (vehicle.type == TRUCK) {}
```

Often violates OCP.

Wrong Inheritance

Using inheritance only for code reuse.

Example:

```
class Stack extends ArrayList {}
```

This can expose behavior that stack should not support.

Often violates LSP.

Fat Interface

One interface contains many unrelated methods.

Often violates ISP.

Hardcoded Dependencies

```
class Service {  
    private MySQLDatabase db = new MySQLDatabase();  
}
```

Often violates DIP.

Final Summary

SOLID is about managing change.

- SRP keeps responsibilities focused.
- OCP allows adding behavior without modifying stable code.
- LSP keeps inheritance and polymorphism correct.
- ISP keeps interfaces small and client-specific.
- DIP keeps high-level business logic independent of low-level details.

In interviews, do not just define SOLID. Use it while explaining your class design, especially around changing requirements such as new payment methods, pricing strategies, notification channels, vehicle types, or storage mechanisms.

The strongest LLD designs usually combine SOLID with simple design patterns, clear domain models, and practical judgment.